# Become a Better Application Tester: A Hands-On Task Manager Guide

Learn and practice different software-testing methods using a Task Manager with a Flutter mobile app, a Node.js/Express REST API, PostgreSQL, and JWT login with refresh tokens.

**Audience:** Beginner to intermediate learners

**Approach:** Test small parts first, then test how they work together and how the whole app behaves. Use local or dedicated test environments only. Never test systems or accounts without permission.

## Practice app and test environment

The app lets a user register, log in, view and manage their own tasks, refresh a session, and log out. The API stores data in PostgreSQL and must prevent one user from accessing another user's tasks.

Before testing, make sure you have:

- A local app and API, plus a separate test database.
- Test accounts and disposable test data.
- A way to view API responses and app logs without exposing tokens or passwords.
- Written permission and a defined scope for any security or load testing.

Keep production data out of tests. Use synthetic accounts, rotate test credentials, and never paste real tokens into reports or screenshots.

## 1. Start with a tester mindset and a plan

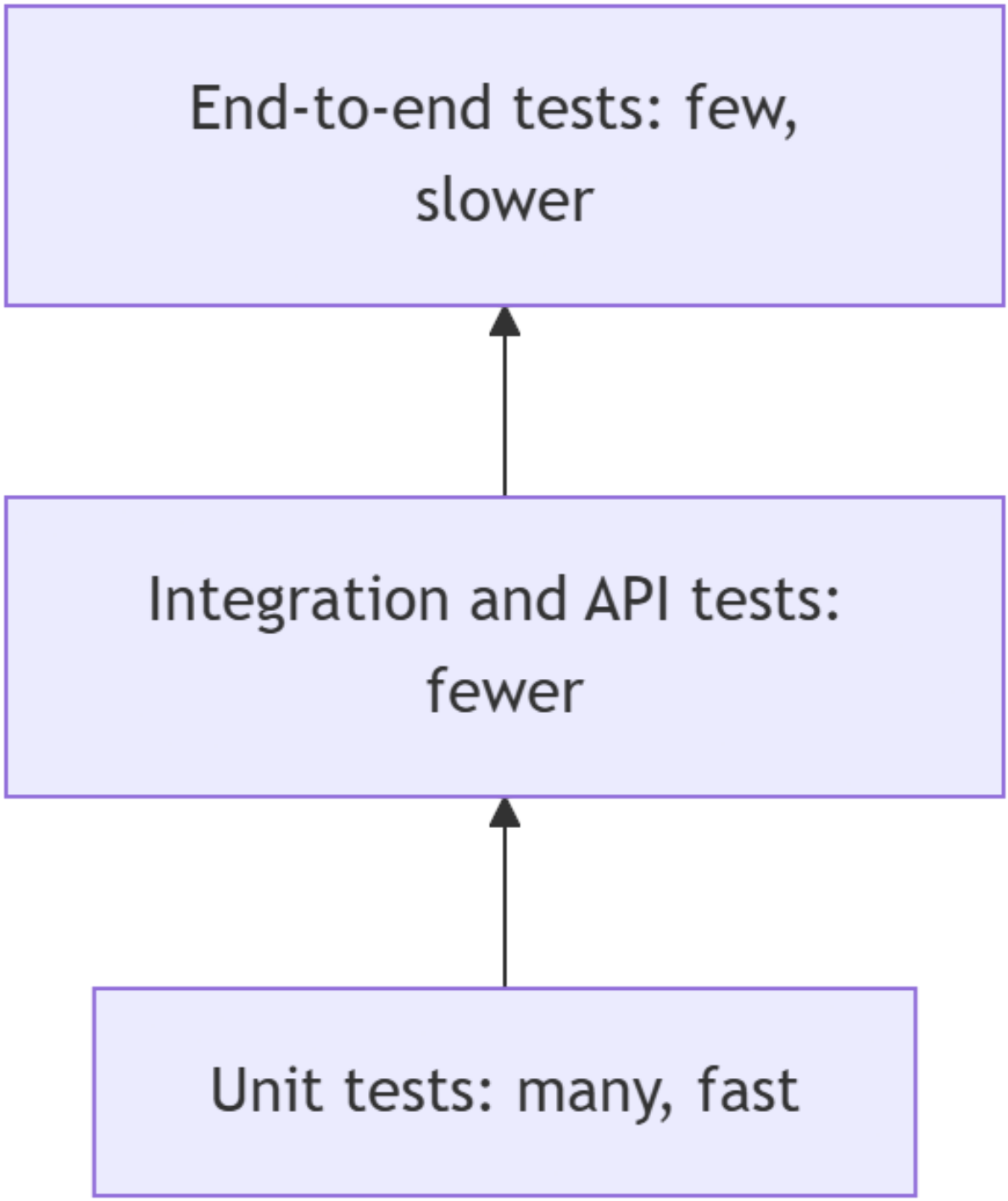
**Goal:** Learn to investigate behavior systematically rather than just confirm that the app works.

**When to use it:** Before a new feature, release, or focused testing session.

**How to do it:**

1. Read the requirement and identify who uses the feature.
2. Write down expected behavior and observable outcomes.
3. Think of normal use, invalid input, edge cases, and risks.
4. Prioritize tests based on user impact and likelihood of failure.
5. Record what you tested, where, and what remains untested.

A useful model is the testing pyramid: many fast, focused checks at the bottom; fewer broader, slower checks at the top.




*Mermaid Diagram*

Example requirement: “A signed-in user can create a task.” Acceptance criteria might say:

- A task with a valid title is saved and appears in the task list.
- A title containing only spaces is rejected with a clear message.
- A user cannot view another account's task by changing the task ID.

**Practice exercise:** Choose one feature, such as task creation. Write three acceptance criteria: one typical success, one invalid input, and one permission boundary.

## 2. Write useful test cases and bug reports

**Goal:** Make tests repeatable and bugs easy for developers to investigate.

**When to use it:** When documenting planned tests or reporting a failure.

**How to do it:** A test case should state its setup, actions, and expected result. A bug report should include enough detail to reproduce the issue without exposing private data.

### Test case template

| Field | What to include |
|---|---|
| ID and title | Short unique ID and behavior being checked |
| Preconditions | Account, app state, or data needed |
| Steps | Numbered actions, one action per step |
| Test data | Synthetic values used |
| Expected result | Observable, specific behavior |
| Actual result | What happened during the run |
| Environment | Device, OS, app version, API build |

### Bug report template

| Field | What to include |
|---|---|
| Summary | Concise description of the problem |
| Environment | Device/OS, app version, API build |
| Preconditions | Account type and relevant setup, using no real personal data |
| Steps to reproduce | Exact steps and safe test data |

| | |
|---|---|
| Expected result | What should happen |
| Actual result | What happened |
| Evidence | Redacted screenshot, logs, or request details |
| Severity | Impact of the defect |
| Priority | How soon it should be fixed |

**Severity** describes impact. **Priority** describes urgency or scheduling. A crash that loses user data can be high severity and high priority. A small visual issue on a rarely used screen may be low severity and low priority. A minor issue blocking an imminent demonstration might have low severity but high priority.

**Practice exercise:** Write a report for a task that disappears after being marked complete. Include steps, expected and actual results, and a severity/priority assessment.

## 3. Explore the app with a charter

**Goal:** Find unexpected behavior by exploring with a question in mind.

**When to use it:** Early in development, after changes, or when requirements are incomplete.

**How to do it:**

1. Choose a focused charter: “Explore task editing to find data-loss problems.”
2. Set a time limit, such as 20 minutes.
3. Try realistic workflows and vary one condition at a time.
4. Record actions, observations, questions, and failures.
5. Turn reproducible failures into test cases and bug reports.

### Session note template

| Time | Action or condition | Observation | Follow-up |
|---|---|---|---|
| 00:00 | Signed in as test user A | Task list loaded | Check empty state |
| 00:05 | Edited a title, then went offline | Save showed an error | Check whether old title remains |

**Practice exercise:** Explore task creation and editing for 15 minutes. Try a long title, a blank title, rapid repeated taps, and briefly interrupted connectivity. Record what surprised you.

## 4. Test units in isolation

**Goal:** Check small functions or classes quickly and precisely.

**When to use it:** Whenever business rules or parsing logic change.

**How to do it:** A unit test supplies controlled inputs and checks a result. Keep it independent of the live network and database.

### Backend example with Jest

Assume the title validation function is in `src/tasks/validation.js`:

```
'use strict';

/**
 * Trims and validates a task title.
 *
 * @param {unknown} value - Untrusted title input.
 * @returns {string} The validated title.
 * @throws {TypeError} When the title is not valid.
 */
function validateTitle(value) {
  if (typeof value !== 'string') {
    throw new TypeError('Enter a valid title.');
  }

  const title = value.trim();
  if (title.length < 1 || title.length > 120) {
    throw new TypeError('Enter a valid title.');
  }

  return title;
}

module.exports = { validateTitle };
```

```
'use strict';

const { validateTitle } = require('../src/tasks/validation');

describe('validateTitle', () => {
  test('trims a valid title', () => {
    expect(validateTitle('  Plan tests  ')).toBe('Plan tests');
  });
```

```
  test('rejects a blank title', () => {
    expect(() => validateTitle('   ')).toThrow('Enter a valid title.');
  });

  test('rejects a title longer than the limit', () => {
    expect(() => validateTitle('a'.repeat(121))).toThrow();
  });
});
```

## Dart model example with flutter_test

```
/// Converts a JSON object into a task value.
class Task {
  /// Creates a task.
  const Task({required this.id, required this.title});

  /// Unique server identifier.
  final String id;

  /// User-visible task title.
  final String title;

  /// Parses a task from JSON.
  ///
  /// Throws [FormatException] when required fields are missing or
invalid.
  factory Task.fromJson(Map<String, dynamic> json) {
    final id = json['id'];
    final title = json['title'];

    if (id is! String || title is! String || title.trim().isEmpty) {
      throw const FormatException('Invalid task data.');
    }

    return Task(id: id, title: title);
  }
}
```

```
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/task.dart';

void main() {
  test('parses a valid task', () {
    final task = Task.fromJson({'id': 'task-1', 'title': 'Plan tests'});

    expect(task.id, 'task-1');
```

```
    expect(task.title, 'Plan tests');
  });

  test('rejects an empty title', () {
    expect(
      () => Task.fromJson({'id': 'task-1', 'title': '  '}),
      throwsFormatException,
    );
  });
}
```

Run backend tests with `npm test` and Flutter tests with `flutter test`.

**Practice exercise:** Add a unit test for a missing ID and one for a valid title at the maximum allowed length.

## 5. Test Flutter widgets

**Goal:** Verify that a widget presents the right screen for loading, empty, error, and data states.

**When to use it:** When changing screen behavior, controls, or user feedback.

**How to do it:** Inject state or dependencies so each screen state can be tested without a real API.

Example widget with a testable list state:

```dart
import 'package:flutter/material.dart';

/// Shows tasks or the current loading, empty, or error state.
class TaskListPanel extends StatelessWidget {
  /// Creates a task list panel.
  const TaskListPanel({
    super.key,
    required this.loading,
    required this.tasks,
    this.errorMessage,
  });

  /// Whether tasks are being loaded.
  final bool loading;

  /// Task titles to display.
  final List<String> tasks;
```

```
  /// Optional safe error message.
  final String? errorMessage;

  @override
  Widget build(BuildContext context) {
    if (loading) {
      return const Center(child: CircularProgressIndicator());
    }
    if (errorMessage != null) {
      return Center(child: Text(errorMessage!));
    }
    if (tasks.isEmpty) {
      return const Center(child: Text('No tasks yet.'));
    }
    return ListView(
      children: [for (final task in tasks) ListTile(title: Text(task))],
    );
  }
}
```

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/task_list_panel.dart';

void main() {
  testWidgets('shows loading state', (tester) async {
    await tester.pumpWidget(
      const MaterialApp(
        home: TaskListPanel(loading: true, tasks: []),
      ),
    );

    expect(find.byType(CircularProgressIndicator), findsOneWidget);
  });

  testWidgets('shows empty state', (tester) async {
    await tester.pumpWidget(
      const MaterialApp(
        home: TaskListPanel(loading: false, tasks: []),
      ),
    );

    expect(find.text('No tasks yet.'), findsOneWidget);
  });

  testWidgets('shows error state', (tester) async {
    await tester.pumpWidget(
```

```
      const MaterialApp(
        home: TaskListPanel(
          loading: false,
          tasks: [],
          errorMessage: 'Could not load tasks.',
        ),
      ),
    );

    expect(find.text('Could not load tasks.'), findsOneWidget);
  });

  testWidgets('shows task data', (tester) async {
    await tester.pumpWidget(
      const MaterialApp(
        home: TaskListPanel(loading: false, tasks: ['Plan tests']),
      ),
    );

    expect(find.text('Plan tests'), findsOneWidget);
  });
}
```

**Practice exercise:** Add a retry button to the error state and test that tapping it invokes a callback exactly once.

## 6. Test the API and integrated backend

**Goal:** Check that routes, middleware, authentication, services, and database access work together.

**When to use it:** When adding endpoints or changing validation, authorization, or persistence.

**How to do it:** Use Supertest against an isolated test app and database. Cover successful requests, invalid input, missing or invalid authentication, and ownership boundaries.

```
'use strict';

const request = require('supertest');
const { app } = require('../src/app');

describe('task API security', () => {
  test('rejects task creation without authentication', async () => {
    const response = await request(app)
      .post('/api/v1/tasks')
      .send({ title: 'Plan tests' });
```

```
    expect(response.status).toBe(401);
  });

  test('rejects an empty title', async () => {
    const token = await createTestUserAndGetToken();

    const response = await request(app)
      .post('/api/v1/tasks')
      .set('Authorization', `Bearer ${token}`)
      .send({ title: '   ' });

    expect(response.status).toBe(400);
  });

  test('does not let one user read another user task', async () => {
    const owner = await createTestUserAndGetToken();
    const stranger = await createTestUserAndGetToken();
    const taskId = await createTask(owner, 'Private task');

    const response = await request(app)
      .get(`/api/v1/tasks/${taskId}`)
      .set('Authorization', `Bearer ${stranger}`);

    expect([403, 404]).toContain(response.status);
  });
});
```

The helper functions are project-specific: create users and tasks in the isolated test database, and clean them up between tests. Use a consistent policy so the response does not disclose whether another user's task exists.

You can also inspect a local endpoint with `curl`:

```
curl -i http://localhost:3000/api/v1/tasks \
  -H "Authorization: Bearer LOCAL_TEST_ACCESS_TOKEN"
```

Do not paste real tokens into shared terminal logs, screenshots, or reports.

**Practice exercise:** Add API tests for an invalid task ID, a missing title, and a user attempting to update another user's task.

## 7. Test complete user journeys

**Goal:** Verify important workflows from the app UI through the API and back.

**When to use it:** Before releases and for a small number of critical user paths.

**How to do it:** Use Flutter's `integration_test` package with a test backend and test account. Keep end-to-end tests focused because they are slower and more sensitive to environment problems.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:my_app/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('user can sign in and create a task', (tester) async {
    app.main();
    await tester.pumpAndSettle();

    await tester.enterText(find.byKey(const Key('email-field')),
'tester@example.test');
    await tester.enterText(find.byKey(const Key('password-field')),
'local-test-password');
    await tester.tap(find.byKey(const Key('login-button')));
    await tester.pumpAndSettle();

    await tester.tap(find.byKey(const Key('add-task-button')));
    await tester.enterText(find.byKey(const Key('task-title-field')),
'Check release flow');
    await tester.tap(find.byKey(const Key('save-task-button')));
    await tester.pumpAndSettle();

    expect(find.text('Check release flow'), findsOneWidget);
  });
}
```

Use dedicated test credentials and reset the test account's data before each run. Never target production.

**Practice exercise:** Extend the journey to mark the task complete, log out, and verify that the login screen appears.

## 8. Use boundary values and equivalence partitions

**Goal:** Select efficient test data that covers meaningful input groups and edge cases.

**When to use it:** For fields with length limits, numeric ranges, or constrained formats.

**How to do it:**

- **Equivalence partitioning:** Group inputs expected to behave alike, then test representative values from each group.
- **Boundary value analysis:** Test values at, just below, and just above a limit.

For a task title that must contain between 1 and 120 characters:

| Partition or boundary | Example input | Expected result |
|---|---|---|
| Empty | `""` | Reject |
| Whitespace only | `" "` | Reject |
| Minimum valid length | `"a"` | Accept |
| Typical valid title | `"Plan tests"` | Accept |
| Maximum valid length | 120 characters | Accept |
| Just over maximum | 121 characters | Reject |
| Wrong type in API JSON | `title: 42` | Reject safely |

**Practice exercise:** Apply this approach to the password field. Check empty, minimum length, typical value, maximum length, and over-limit input. Avoid testing with a real password.

## 9. Test negative behavior and security basics

**Goal:** Check that the app rejects invalid or unauthorized actions safely.

**When to use it:** For every endpoint that accepts user input, requires authentication, or accesses private data.

**How to do it:**

1. Send invalid types, missing fields, and oversized values. Confirm safe validation errors.
2. In a local test environment, try common injection-looking strings, such as a title containing `' OR '1'='1`. Confirm the data layer treats it as ordinary input, not executable SQL.
3. For text shown in the UI, test markup-like input such as `<script>alert(1)</script>`. Confirm it is displayed as text and is not executed or interpreted as unsafe markup.
4. Test access control by using User B's token with User A's task ID.

5. Test rate limits using a small, authorized test run. Confirm the API returns a safe rate-limit response.
6. Test expired access tokens, refresh rotation, reuse detection, logout, and logout-all.

Do not run destructive payloads or security scans against systems you do not own or have permission to test.

### OWASP API security checklist

| Concern | Task Manager check |
| --- | --- |
| Broken object-level authorization | Every task query is scoped to the authenticated owner |
| Broken authentication | Invalid, expired, and revoked tokens are rejected |
| Broken object property-level authorization | Clients cannot set owner IDs or privileged fields |
| Unrestricted resource consumption | Pagination, body size, and request limits are bounded |
| Broken function-level authorization | Protected routes require appropriate identity and role |
| Unrestricted access to sensitive business flows | Login and refresh attempts are rate-limited |
| Server-side request forgery | No user-controlled upstream URL is accepted |
| Security misconfiguration | Production uses HTTPS, safe headers, and restricted CORS |
| Improper inventory management | Document active API versions and remove old routes |
| Unsafe consumption of APIs | Validate and safely handle external API responses |

**Practice exercise:** In the test environment, try to update a task with an `ownerId` field. Verify that the API ignores or rejects it and retains server-derived ownership.

## 10. Check performance and load safely

**Goal:** Understand response time and service behavior under a planned workload.

**When to use it:** Before launch or when a performance concern is reported.

**How to do it:** Get approval, use a dedicated environment, start with a small load, and set a clear stop condition. Measure response time, error rate, throughput, CPU, memory, and database connections.

Example k6 script for an authorized test environment:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 5,
  duration: '30s',
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<500'],
  },
};

export default function () {
  const response = http.get(`${__ENV.BASE_URL}/health`);

  check(response, {
    'health returns 200': (r) => r.status === 200,
  });

  sleep(1);
}
```

Run only against a system you control:

```
BASE_URL=http://localhost:3000 k6 run health-test.js
```

Do not load-test production without written authorization, coordination, and a rollback/stop plan.

**Practice exercise:** Run the health check against your local API. Change one threshold, rerun, and explain what the report says.

## 11. Check usability and accessibility

**Goal:** Confirm that different users can understand and operate the app.

**When to use it:** During feature development and before release.

**How to do it:** Test with platform accessibility tools and real devices or emulators where possible.

### Quick checklist

- Text and controls have sufficient contrast.
- Buttons and fields have clear labels and meaningful accessible names.
- Screen readers announce task titles, status, and errors in a useful order.
- The app remains usable at larger text sizes.
- Touch targets are easy to select.
- Errors explain how to recover.
- Color is not the only way to communicate status.
- Keyboard and switch navigation are considered where applicable.

**Practice exercise:** Enable a screen reader and larger text size. Create a task, trigger a validation error, and record any unlabeled or clipped controls.

## 12. Test device and platform compatibility

**Goal:** Find issues that appear only on specific devices or operating-system versions.

**When to use it:** Before a mobile release and after changes to layout, networking, storage, or permissions.

**How to do it:** Build a small matrix based on user distribution and risk. You do not need every device; choose representative combinations.

| Platform | Device profile | OS range | Checks |
|---|---|---|---|
| Android | Small screen | Supported minimum | Login, text scaling, network |
| Android | Current mainstream device | Current OS | Task CRUD, refresh, logout |
| iOS | Small screen | Supported minimum | Layout, secure storage, accessibility |
| iOS | Current mainstream device | Current OS | Login, lifecycle, network recovery |
| Emulator/simulator | Development profile | Current toolchain | Fast automated smoke checks |

**Practice exercise:** Test the task list on a small screen and a larger screen. Note overflow, clipped text, and controls that are hard to tap.

## 13. Run smoke tests, regression tests, and manage test data

**Goal:** Quickly check critical health after a build and ensure existing behavior still works after changes.

**When to use it:**

- **Smoke test:** After deployment or a new build; verify basic critical functions.
- **Regression test:** After code changes; check existing features that could be affected.

**How to do it:**

1. Keep a short smoke suite: app opens, login works, task list loads, a task can be created, logout works.
2. Run focused regression tests for changed areas.
3. Keep test data synthetic and reproducible.
4. Reset only a local or dedicated test database, never a shared or production database.
5. Record test environment and build so results can be reproduced.

**Practice exercise:** Make a regression checklist for task creation, editing, completion, deletion, and cross-user access.

## 14. Automate tests in CI and understand coverage

**Goal:** Run repeatable checks on each change and catch failures before merging.

**When to use it:** As soon as a project has tests and a shared repository.

**How to do it:** A GitHub Actions workflow can run backend and Flutter tests. Adapt paths, versions, and database setup to your project.

```yaml
name: Tests

on:
  pull_request:
  push:
    branches: [main]

jobs:
```

```yaml
  backend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: backend/package-lock.json
      - run: npm ci
      - run: npm test

  flutter:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: mobile
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
      - run: flutter pub get
      - run: flutter test
```

Coverage indicates which code ran during tests; it does not prove that tests checked the right behavior. A high percentage can still miss important permissions, edge cases, or user journeys.

**Practice exercise:** Add one test to the backend job and verify that CI fails when the expected behavior is intentionally broken, then restore the code.

## 15. Choose mocks carefully and manage flaky tests

**Goal:** Decide when to isolate dependencies and when to test real integrations.

**When to use it:** When tests rely on HTTP, a database, time, or platform services.

**How to do it:**

- Use mocks for fast, deterministic unit and client tests.
- Use a real isolated test database for important persistence and transaction behavior.
- Keep a small number of end-to-end tests for critical journeys.

- Avoid depending on live third-party services in routine test runs.
- Fix flaky tests by controlling time, data, network, and ordering rather than repeatedly rerunning them.
- Never mock the behavior you are trying to verify; for example, an ownership test must exercise the authorization boundary.

**Practice exercise:** Compare one mocked API-client test with one API integration test using a dedicated test database. Record what each catches and what each cannot catch.

## Compare common test types

| Test type | Purpose | Who commonly runs it | Typical tools | Relative speed |
|---|---|---|---|---|
| Unit | Verify a small function or class | Developer, QA engineer | Jest, `flutter_test` | Very fast |
| Widget/component | Verify a UI component and states | Developer, QA engineer | `flutter_test` | Fast |
| API/integration | Verify route, middleware, service, and persistence together | Developer, QA engineer | Supertest, `curl`, test database | Medium |
| End-to-end/UI | Verify a complete user journey | QA engineer, developer | Flutter `integration_test` | Slow |
| Exploratory | Discover unexpected behavior through guided investigation | Tester, developer, product team | App, notes, device tools | Variable |
| Security | Check authorization, validation, and safe failure behavior | Trained tester, security specialist | Tests, approved scanners, OWASP guidance | Variable |
| Performance/loa | Measure | Performance | k6 | Variable |

| | | | | |
|---|---|---|---|---|
| d | behavior under an agreed workload | tester, developer | | |
| Accessibility/usability | Assess access and ease of use | Tester, users, accessibility specialist | Screen readers, platform tools | Variable |
| Compatibility | Check behavior across devices and OS versions | QA engineer, device lab | Real devices, simulators | Variable |
| Smoke/regression | Confirm critical behavior and detect unintended changes | Developer, QA engineer | CI and test suites | Depends on scope |

## Test plan template

| Field | Notes |
|---|---|
| Feature/release | What is being tested |
| Scope | Included and excluded behavior |
| Risks | User, security, data, and availability risks |
| Test environment | App/API build, database, devices, OS |
| Test data | Synthetic users and reset steps |
| Test types | Unit, widget, API, end-to-end, exploratory, etc. |
| Entry criteria | What must be ready before testing |
| Exit criteria | What must pass before completion |
| Owners | Who tests, fixes, and approves |
| Evidence | Where redacted results and reports are stored |

## Tester skills checklist

Turn requirements into observable acceptance criteria.

Write reproducible test cases and clear bug reports.

Explore with a focused charter and take useful notes.

Choose representative boundary and negative inputs.

Understand authentication versus authorization.

Test API responses and app states independently.

Protect credentials and personal data in test evidence.

Communicate risk and uncertainty clearly.

Use automation where it saves repeatable effort.

Ask for permission and scope before security or load testing.

## Four-week practice plan

| Week | Focus | Practice outcome |
|---|---|---|
| 1 | Mindset, plans, test cases, exploratory testing | A test plan, three test cases, and one exploratory report |
| 2 | Unit, widget, API, and integration tests | Tests for title validation, task states, and ownership |
| 3 | End-to-end, boundaries, security, accessibility | One journey, boundary table, and security/accessibility notes |
| 4 | Performance, compatibility, regression, CI | Local k6 run, device matrix, regression checklist, CI workflow |

## Common mistakes

- Testing only the happy path.
- Writing vague expected results such as “works correctly.”
- Treating a passing test as proof that the feature is secure.
- Using real user data or credentials in test environments.
- Testing ownership with only one account.
- Including tokens, passwords, or private data in screenshots and logs.

- Writing end-to-end tests for every detail instead of reserving them for important workflows.
- Ignoring flaky failures or repeatedly rerunning them without finding the cause.
- Running load or security tests without permission and a defined scope.
- Treating code coverage as a measure of test quality.

## Troubleshooting

- **Tests fail only on a developer's machine:** Check dependency versions, environment variables, timezone, and test data setup.
- **API tests affect each other:** Give each test isolated data and clean it up reliably.
- **Widget test hangs:** Check for pending animations, unresolved futures, or missing `pumpAndSettle` constraints.
- **End-to-end test cannot sign in:** Confirm the test API URL, test account, and test database state.
- **Intermittent failure:** Capture logs safely, control network/time dependencies, and check whether tests run in parallel against shared data.
- **Unexpected 401:** Check token expiry, refresh behavior, and that the latest token pair was stored.
- **Unexpected cross-user access:** Stop testing against shared data, preserve safe evidence, and report the access-control issue promptly through the approved process.

## Official references

- Jest documentation
- Supertest
- Flutter testing overview
- Flutter integration testing
- k6 documentation
- OWASP API Security Top 10
- ISTQB glossary
- Dart documentation comments

Official education standards relevant to this tutorial could not be verified, so no specific standard codes are listed.